

Next.js Lesson 5 — Continue

We left off at `useFormStatus` .

You already understand the important pieces:

- `analyzeWebsite` → the **Server Action**
- `useActionState(analyzeWebsite, initialState)` → connects that action to React state
- `formAction` → the **function returned by** `useActionState` that you give to `<form action={formAction}>`
- When the form submits → `formAction` runs → internally it invokes `analyzeWebsite`

Now the next question is:

How do I know whether that `formAction` is currently running?

That's exactly what `useFormStatus` solves. 📌

5.1 — `useFormStatus`

Think of it as:

```
useActionState
  ↓
gives us formAction
  ↓
<form action={formAction}>
  ↓
user submits
  ↓
formAction is running
  ↓
useFormStatus()
```



↓
pending = true

So:

```
const { pending } = useFormStatus();
```

means:

"Is the form submission that this component belongs to currently in progress?"

Example

```
function SubmitButton() {  
  const { pending } = useFormStatus();  
  
  return (  
    <button disabled={pending}>  
      {pending ? "Analyzing..." : "Analyze Website"}  
    </button>  
  );  
}
```

Now the UI automatically becomes:

Before submit:

[Analyze Website]

↓ user clicks

During Server Action:

[Analyzing...] ← disabled

↓ action finishes

[Analyze Website]

⚠ One VERY important rule

`useFormStatus()` needs to be used **inside the form**.

For example:

```
<form action={formAction}>
  <input name="url" />

  <SubmitButton />
</form>
```

and:

```
function SubmitButton() {
  const { pending } = useFormStatus();

  return (
    <button disabled={pending}>
      {pending ? "Analyzing..." : "Analyze Website"}
    </button>
  );
}
```

Why separate `SubmitButton` ?

Because `useFormStatus()` reads the status of the parent `<form>` .

So conceptually:

```
<form action={formAction}>
  |
  | "Am I submitting?"
  ↓
<SubmitButton />
  |
  ↓
useFormStatus()
  |
  ↓
pending
```



The simplest mental model

Don't mix these two hooks:

`useActionState`

"Give me the **result/state** of my Server Action."

`useFormStatus`

"Tell me whether the **form submission is currently running**."

That's the whole idea. 🎯

Next we'll connect `useActionState` + `useFormStatus` **together** in the actual website analyzer flow, because that's where this becomes really intuitive.



ok

Perfect 👍 Let's put the two hooks **side by side**, because this is where the confusion usually disappears.

 `useActionState` + `useFormStatus`

Imagine our analyzer:

```
function AnalyzerForm() {
  const [state, formAction] = useActionState(
    analyzeWebsite,
    initialState
  );

  return (
    <form action={formAction}>
      <input name="url" />
      <SubmitButton />
    </form>
  );
}
```

And the button:

```
function SubmitButton() {
```

`</>` JavaScript



```

return (
  <button disabled={pending}>
    {pending ? "Analyzing..." : "Analyze Website"}
  </button>
);
}

```

Now watch what happens.

Step 1 — Initial render

```

AnalyzerForm
├── state = initialState
├── formAction = returned function
└── <form action={formAction}>
    └── SubmitButton
        └── pending = false

```

So the button says:

```
[ Analyze Website ]
```

Step 2 — User submits

The browser submits the form:

```

<form action={formAction}>
  ↓
  formAction()
  ↓
  analyzeWebsite()
  ↓
  Server is working...

```

At this exact moment:

```

</> JavaScript
pending === true

```

So:

[Analyzing...] ← disabled



Step 3 — Server Action finishes

Suppose `analyzeWebsite` returns:

```
</> JavaScript
{
  success: true,
  score: 92
}
```



`useActionState` receives that result:

```
state
↓
{
  success: true,
  score: 92
}
```



And the form is no longer submitting:

```
</> JavaScript
pending === false
```



So:

[Analyze Website]



and somewhere else we can display:

Performance Score: 92



The key distinction

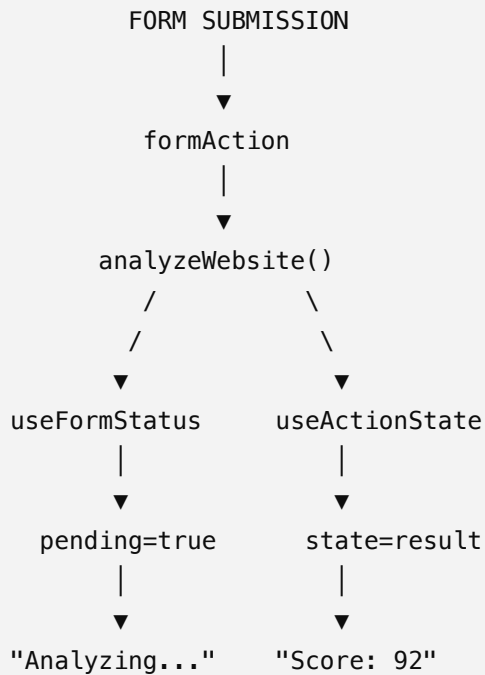
Think of them as answering **two different questions**:

Hook

Question

`useActionState`**"What did my Server Action return?"**`useFormStatus`**"Is my form submission still running?"**

So:



🔥 This is the mental model I want you to keep.

`pending` is about the **current operation**.

`state` is about the **result of the operation**.

And importantly, `useFormStatus` **doesn't invoke the Server Action and doesn't replace** `formAction`. It simply observes the form submission status.

Next concept: **why** `useFormStatus` **is usually put into a separate child component instead of calling it directly beside** `useActionState`.



Exactly 👍 This little rule is **super important**.

Why does `useFormStatus` need a child component?

You might naturally try this:

`</>` JavaScript



```
function AnalyzerForm() {
  const [state, formAction] = useActionState(
    analyzeWebsite,
    initialState
  );

  const { pending } = useFormStatus(); // ❌

  return (
    <form action={formAction}>
      <input name="url" />

      <button disabled={pending}>
        {pending ? "Analyzing..." : "Analyze Website"}
      </button>
    </form>
  );
}
```

It looks perfectly reasonable.

But `pending` won't represent this form's submission.

Why?

Because `useFormStatus()` looks **upward** for a `<form>`.

It basically asks:



"Is the nearest parent form currently submitting?"

But here:

```
AnalyzerForm
├──
├── useFormStatus() ← I'm here
├──
└── <form>
```



The `<form>` is **below** the hook.

There is no parent form for `useFormStatus` to observe.

So we create a child 📌

```

</> JavaScript
function AnalyzerForm() {
  const [state, formAction] = useActionState(
    analyzeWebsite,
    initialState
  );

  return (
    <form action={formAction}>
      <input name="url" />

      <SubmitButton />
    </form>
  );
}

```

Then:

```

</> JavaScript
function SubmitButton() {
  const { pending } = useFormStatus();

  return (
    <button disabled={pending}>
      {pending ? "Analyzing..." : "Analyze Website"}
    </button>
  );
}

```

Now the component tree is:

```

AnalyzerForm
├── <form>
│   ├── <input />
│   └── SubmitButton
│       └── useFormStatus()

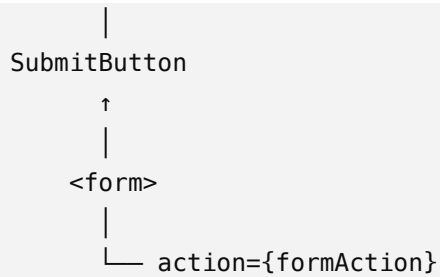
```

Now `useFormStatus()` can look **upward**:

```

useFormStatus()
  ↑

```



So when that form is submitting:

```
</> JavaScript
```

```
pending === true
```



Think of it like this

`useFormStatus()` is basically a **child asking its parent**:

 "Hey `<form>`, are you currently submitting?"

The parent form says:

`<form>` → "Yep, I'm submitting!"

And the child gets:

```
</> JavaScript
```

```
pending === true
```



That's why this pattern is so common:

```

<form>
  ↓
<SubmitButton>
  ↓
useFormStatus()
  
```



One more important detail

`useFormStatus` isn't specifically about **Server Actions**.

It's about the **form submission status**.

So its job is much narrower:

```
useActionState  
→ manages Action + returned state  
  
useFormStatus  
→ observes Form + submission status
```



Once you see that separation, the two hooks stop feeling like they're doing the same thing. 😊

Next: we'll look at the other values `useFormStatus()` gives you besides `pending` — especially `data`, `method`, and `action`.



ok

Great 😊 Let's finish `useFormStatus` properly.

`useFormStatus()` gives more than `pending`

You can write:

```
</> JavaScript  
  
const {  
  pending,  
  data,  
  method,  
  action  
} = useFormStatus();
```



There are **4 pieces of information**.

1. `pending` ★ — the one you'll use most

This tells you:

"Is this form currently submitting?"

```
</> JavaScript  
  
const { pending } = useFormStatus();
```



So:

```
</> JavaScript

<button disabled={pending}>
  {pending ? "Analyzing..." : "Analyze Website"}
</button>
```

This is the main use case.

2. data

`data` gives you access to the **form data** being submitted.

Suppose:

```
</> JavaScript

<form action={formAction}>
  <input name="url" />
  <button>Analyze</button>
</form>
```

If the user enters:

`https://google.com`

then while submitting, `data` represents roughly:

`url → https://google.com`

You can access it through:

```
</> JavaScript

data.get("url")
```

So you could theoretically do:

```
</> JavaScript

const { data } = useFormStatus();

const url = data?.get("url");
```

But honestly, in most applications, **you won't need this.**

3. method

This tells you the HTTP method used by the form.

For example:

```
</> JavaScript
<form method="post">
```



Then:

```
</> JavaScript
method === "post"
```



If you don't specify a method, the default is generally:

```
get
```



Again, this isn't something you'll commonly use with Server Actions.

4. action

This tells you about the form's current action.

In our example:

```
</> JavaScript
<form action={formAction}>
```



`useFormStatus()` can expose that action.

But here's the important distinction:

Don't confuse these two:

```
formAction
```



from:

```
</> JavaScript
```



```
useActionState(...)
```

is the **function** you give to the form.

Whereas:

```
action
```



from:

```
</> JavaScript
```



```
useFormStatus()
```

is **information** about the action associated with the currently submitting form.

But what should YOU actually remember?

For our Next.js application, don't overload your brain with all four.

Think:

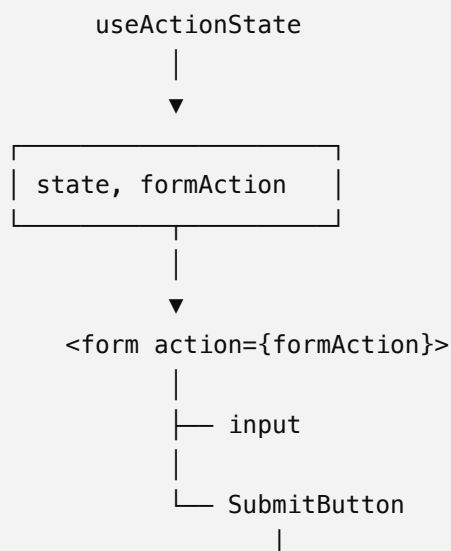
```
</> JavaScript
```

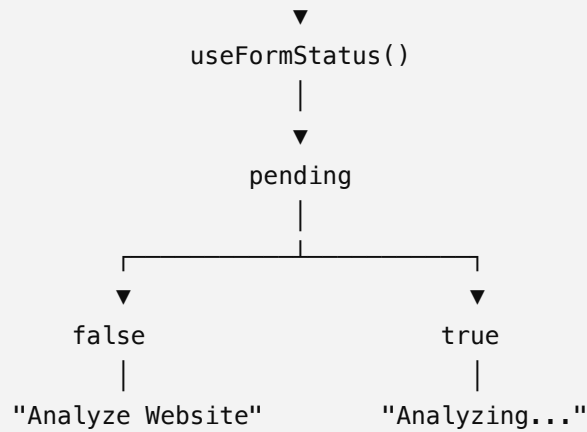


```
const { pending } = useFormStatus();
```

90% of the time, this is what you're after.

Your architecture is:





So now you've got the **complete mental model** of `useFormStatus` . 🧠✅

Lesson 5 so far

Mutations
 ↓
 Server Actions
 ↓
 "use server"
 ↓
 useActionState
 ↓
 useFormStatus ← WE ARE HERE



The next important piece is **revalidation** — after a mutation changes data, how does Next.js know which cached/server-rendered data needs to be refreshed?

That's where `revalidatePath()` and `revalidateTag()` come in. 🚀



wait so data we get in useFormStatus..the data we submitted or result we get

The data you submitted. ✅ Not the result from the Server Action.

This distinction is really important.

Think of the flow:

User fills form
 ↓
 url = "https://google.com"



```
↓
form submitted
↓
useFormStatus()
  |
  └─ data → what was submitted
        ↓
        url = google.com

↓
Server Action runs
↓
analyzeWebsite()
↓
returns result
↓
useActionState()
  |
  └─ state → result from Server Action
```

So:

Hook	Value	Meaning
<code>useFormStatus().data</code>	FormData	 What you submitted
<code>useActionState().state</code>	Whatever your action returns	 What the Server Action returned

Example

You submit:

URL: `https://google.com`



Then during submission:

`</>` JavaScript



```
const { data } = useFormStatus();

data.get("url")
// → "https://google.com"
```

But your Server Action might return:

`</>` JavaScript




```
return {  
  success: true,  
  score: 95  
};
```

That goes into:

JavaScript



```
const [state, formAction] = useActionState(...);  
  
state  
// → { success: true, score: 95 }
```

🔥 Super simple memory trick

useFormStatus → 📤 going OUT

useActionState → 📥 coming BACK

That's the clean distinction.



ChatGPT can make mistakes. Check important info.